



Práctica Extra 3

Organización y Arquitectura de Computadoras

Docente: M.C. Garcia Rocha Jose Isabel

Alumno:

- Chaparro Herrera Hugo Giovanni - 2200357

Tijuana, Baja California a 30 de noviembre de 2025

Objetivo

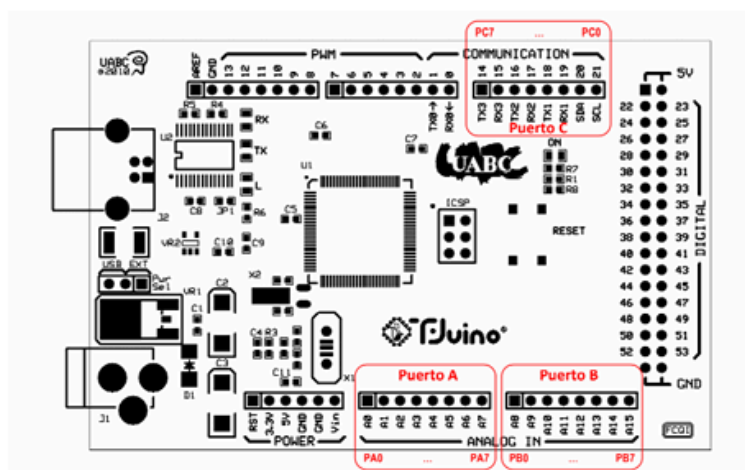
Seleccionar las instrucciones correctas en aplicaciones de sistemas basados en microprocesador mediante la distinción de su funcionamiento, de forma lógica y responsable.

Desarrollo

```

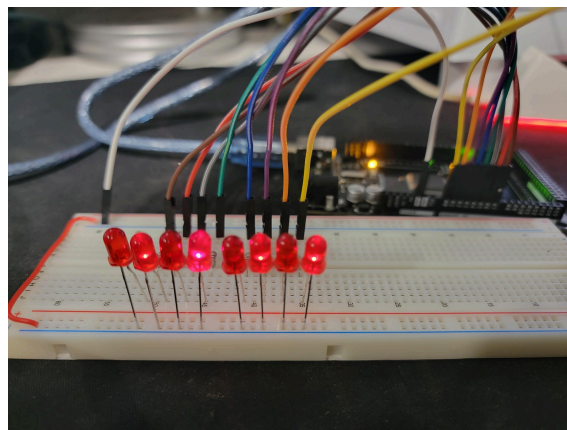
1  .model tiny
2  locals
3  .data
4      PA          DW 0040h
5      PB          DW 0041h
6      PC          DW 0042h
7      RCtrl       DW 0043h
8      PTOs_all_out DB 80h ;80h para indicar que todos los puertos son de salida
9
10 .code
11     org 100h
12
13 principal proc
14 .rep:
15     mov sp, 0fffh
16
17     mov dx, RCtrl ;en dx recibe el puerto
18     mov al, PTOs_all_out ;en al el dato a sacar
19     call outportb
20
21
22     mov dx, PA ;en dx recibe el puerto
23     mov al, 55h ;en al el dato a sacar
24     call outportb
25
26     jmp .rep
27
28     ret
29
30 endp
31 outportb proc
32     ;en dx recibe el puerto
33     ;en al el dato a sacar
34     out dx, al
35
36     ret
37
38 endp
39 end principal

```



1. Realizar lo siguiente:
 - a. Copiar el código del ejemplo de puertos en el archivo P12_Prueba.asm
 - b. Ensamblar y enlazar el archivo para obtener el P12_Prueba.com
 - c. Cargar el archivo P12_Prueba.com a la placa T-Juino
 - d. Ir trazando la ruta del código ensamblador con 't', para ir revisando como va actuando en los registros y los puertos el código de ejemplo.
 - e. Al mismo tiempo poner leds en la salida del puerto para ver como saca el valor indicado por el código P12_Prueba.asm

```
1 .model tiny
2 locals
3 .data
4     PA                DW 0040h
5     PB                DW 0041h
6     PC                DW 0042h
7     RCtrl             DW 0043h
8     PTOs_all_out      DB 80h ; 80h Indica que todos los puertos son de salida
9
10 .code
11     org 100h
12
13 principal proc
14     mov sp, 0fffh
15
16     mov dx, 43h        ; RCtrl
17     mov al, 80h        ; PTOs_all_out
18     call outportb
19
20     mov dx, 40h        ; DX contiene el puerto (En este caso, PA)
21     mov al, 55h        ; AL contiene el dato a sacar por el puerto
22     call outportb
23 .rep:
24     jmp .rep
25     ret
26 endp
27
28 outportb proc ; DX recibe el puerto | AL el dato a sacar
29     out dx, al
30     ret
31 endp
32 end principal
```





2. Cree un programa que contenga las siguientes rutinas para el intérprete 80x86:
a) **SetBit**: Activa un bit del registro AX. El número de bit a activar está dado por CL.

Para la creación de esta rutina, se inicia BX en 1 para que nos sirva como máscara a aplicar con AX, iremos recorriendo el 1 CL veces con corrimientos a la izquierda y aplicaremos un OR (AX | BX) para activar el bit elegido.

```

36 setBit PROC
37     push bx          ; se guarda BX en la pila
38     mov bx, 1        ; se coloca 1 en BX para hacer la mascara
39     shl bx, cl       ; se recorre a la izquierda CL veces
40
41     or ax, bx        ; se aplica la mascara en AX
42     pop bx           ; se restaura el valor de bx
43     ret
44 ENDP

```

```

##### OK!
>9
00001100

```

- b) **ClearBit**: desactiva un bit del registro AX. El número de bit a desactivar está dado por CL.

Para la creación de esta rutina, se inicia BX en 1 para que nos sirva como máscara a aplicar con AX, iremos recorriendo el 1 CL veces con corrimientos a la izquierda y negaremos BX con la finalidad de tener en 0 el bit a apagar, por ultimo, con una AND de (AX & BX) forzaremos el apagado de dicho bit.

```

46 clearBit PROC
47     push bx          ; se guarda BX en la pila
48     mov bx, 1        ; se coloca 1 en BX para hacer la mascara
49     shl bx, cl       ; se recorre a la izquierda CL veces
50     not bx           ; se niega BX para tener en 0 el bit que queremos apagar
51
52     and ax, bx        ; se aplica la mascara en AX
53     pop bx           ; se restaura el valor de bx
54     ret
55 ENDP

```

```

>9
00001100
00001000
-----

```



c) **NotBit**: Invierte un bit del registro AX. El número de bit a invertir está dado por CL.

Mismo caso, recorremos CL veces y aplicamos un XOR ($AX \wedge BX$) para que el 1 invierta el valor que queremos negar.

```
57 notBit PROC
58     push bx        ; se guarda BX en la pila
59     mov bx, 1      ; se coloca 1 en BX para hacer la mascara
60     shl bx, cl     ; se recorre a la izquierda CL veces
61
62     xor ax, bx     ; se aplica la mascara en AX
63     pop bx         ; se restaura el valor de bx
64     ret
65 ENDP
```

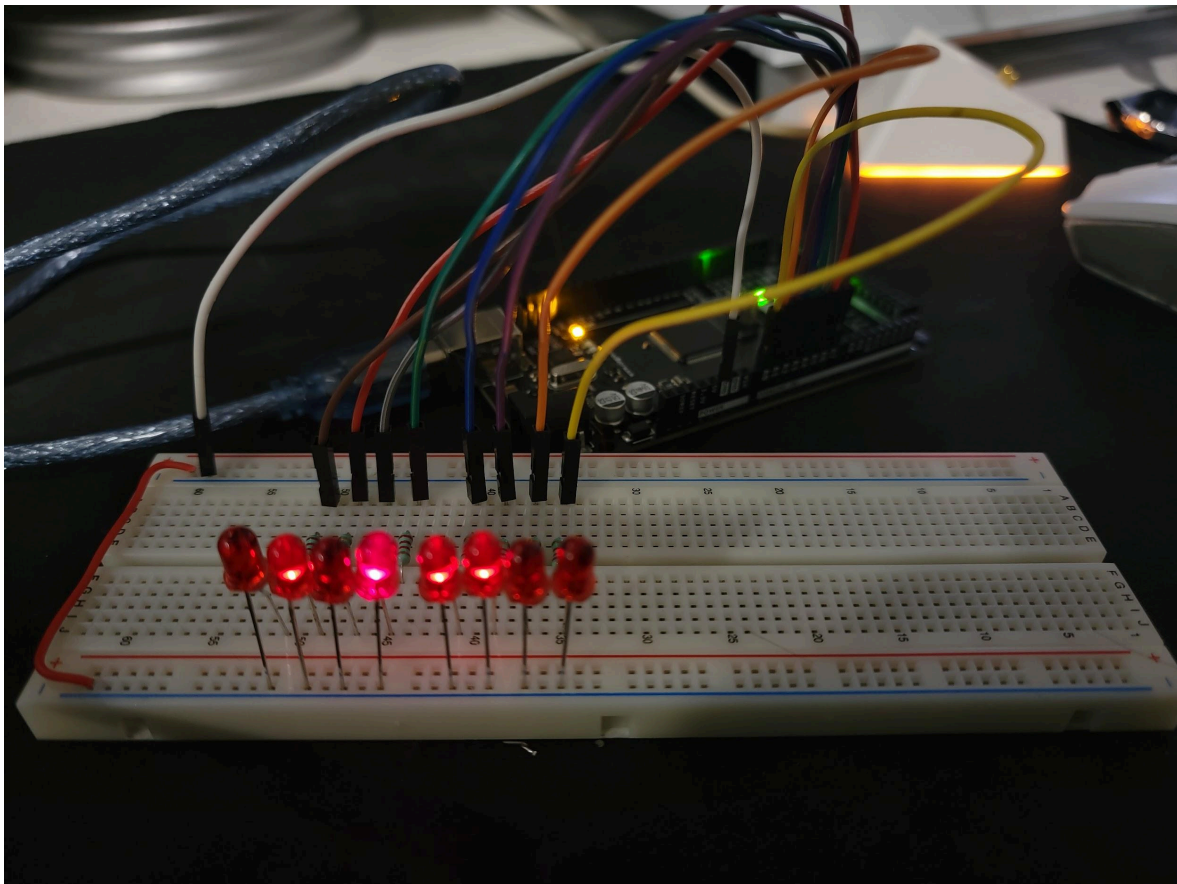
```
>g
00001100
00001000
00001001
```

3. Realizar las siguientes subrutinas en ensamblador para la placa T-Juino.

- SetBitPort: Manipula la información de un puerto dado por DX para activar un determinado bit. Es decir, mediante ella se puede activar (hacer uno) un bit del puerto. El número del bit está en el rango de 0 a 7 siendo el bit 7 es más significativo y está dado por CL.

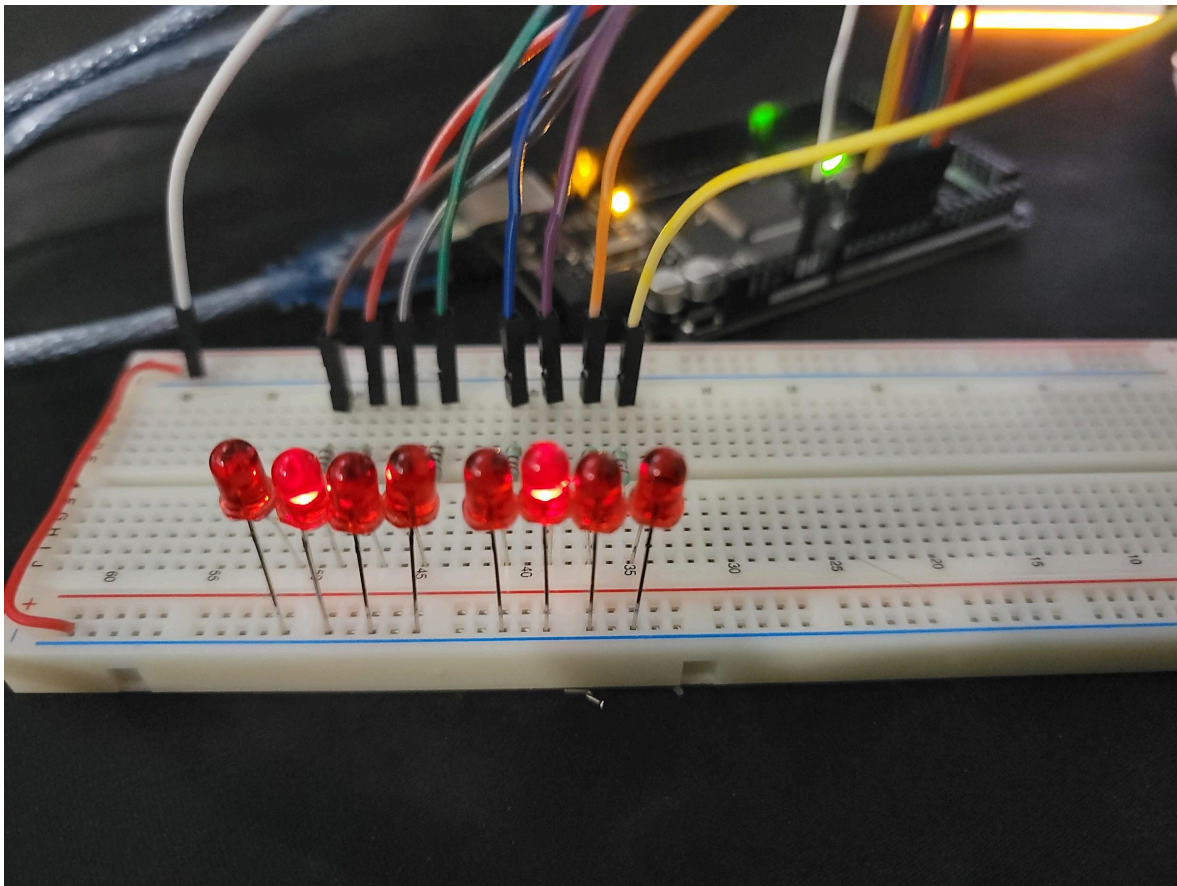
Mismo caso que las funciones anteriores, solamente se le agrega un in y un out para la extracción del dato.

```
107 setBitPort PROC
108     push bx      ; se guarda BX en la pila
109     in al, dx
110     mov bx, 1    ; se coloca 1 en BX para hacer la mascara
111     shl bx, cl   ; se recorre a la izquierda CL veces
112
113     or ax, bx    ; se aplica la mascara en AX
114     out dx, al
115     pop bx       ; se restaura el valor de BX
116     ret
117 ENDP
```



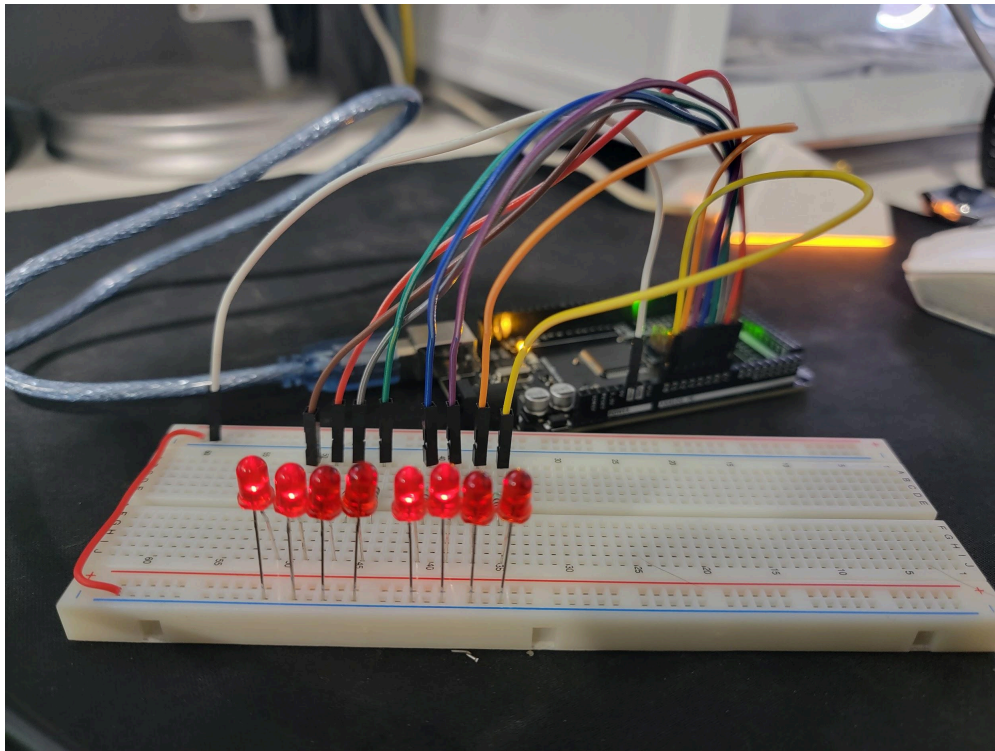
- ClearBitPort: Manipula la información de un puerto dado por DX para desactivar un determinado bit. Es decir, mediante ella se puede desactivar (hacer cero) un bit del puerto. El número del bit está en el rango de 0 a 7 siendo el bit 7 es más significativo y está dado por CL.

```
119 clearBitPort PROC
120     push bx        ; se guarda BX en la pila
121     in al, dx
122     mov bx, 1      ; se coloca 1 en BX para hacer la mascara
123     shl bx, cl     ; se recorre a la izquierda CL veces
124     not bx         ; se niega BX para tener en 0 el bit que queremos apagar
125
126     and ax, bx     ; se aplica la mascara en AX
127     out dx, al
128     pop bx        ; se restaura el valor de BX
129     ret
130 ENDP
```



- NotBitPort: Manipula la información de un puerto dado por DX para invertir un determinado bit. Es decir, mediante ella se puede invertir un bit del puerto. El número del bit está en el rango de 0 a 7 siendo el bit 7 es más significativo y está dado por CL.

```
132 notBitPort PROC
133     push bx      ; se guarda BX en la pila
134     in al, dx
135     mov bx, 1    ; se coloca 1 en BX para hacer la mascara
136     shl bx, cl   ; se recorre a la izquierda CL veces
137
138     xor ax, bx   ; se aplica la mascara en AX
139     out dx, al
140     pop bx      ; se restaura el valor de BX
141     ret
142 ENDP
```





Código en ASM

```
.model tiny
locals
.data
    PA            DW 0040h
    PB            DW 0041h
    PC            DW 0042h
    RCtr          DW 0043h
    PTOs_all_out  DB 80h ; 80h Indica que todos Los puertos son de salida

.code
    org 100h

    principal PROC
    mov sp, 0ffffh

    mov dx, 43h ; RCtr
    mov al, 80h ; PTOs_all_out
    call outportb

    ; 2. a)
    mov ax, 4h
    mov cl, 3
    call setBit
    call printbin ; debe mostrar 0000 1100
    call salto

    ; 2. b)
    mov cl, 2
    call clearBit
    call printbin ; debe mostrar 0000 1000
    call salto

    ; 2. c)
    mov cl, 0
    call notBit
    call printbin ; debe mostrar 0000 1001
    call salto

    ; - - - - -
    ; 3.
    mov dx, 40h
    mov al, 4Ch
    out dx, al
    in al, dx
    call printbin ; debe imprimir 0100 1100

    ; 3. a)
    ; mov dx, 40h
```



```
; mov cl, 4
; call setBitPort ; debe mostrar 0101 0100

; 3. b)
mov dx, 40h
mov cl, 3
call clearBitPort ; debe mostrar 0100 0100

; 3. c)
; mov dx, 40h
; mov cl, 7
; call notBitPort ; debe mostrar 1100 1100

@@fin:
    jmp @@fin
    ret
    endp

; - - - - -
outportb proc ; DX recibe el puerto | AL el dato a sacar
    out dx, al
    ret
endp

; - - - - -

setBit PROC
    push cx      ; se guarda BX en la pila
    mov bx, 1    ; se coloca 1 en BX para hacer la mascara
    shl bx, cl   ; se recorre a la izquierda CL veces

    or ax, bx    ; se aplica la mascara en AX
    pop cx       ; se restaura el valor de bx
    ret
ENDP

clearBit PROC
    push cx      ; se guarda BX en la pila
    mov bx, 1    ; se coloca 1 en BX para hacer la mascara
    shl bx, cl   ; se recorre a la izquierda CL veces
    not bx       ; se niega BX para tener en 0 el bit que queremos apagar

    and ax, bx   ; se aplica la mascara en AX
    pop cx       ; se restaura el valor de bx
    ret
ENDP

notBit PROC
    push cx      ; se guarda BX en la pila
    mov bx, 1    ; se coloca 1 en BX para hacer la mascara
```



```
    shl bx, cl ; se recorre a la izquierda CL veces

    xor ax, bx ; se aplica la mascara en AX
    pop cx     ; se restaura el valor de bx
    ret
ENDP

; - - - - -
setBitPort PROC
    push bx ; se guarda BX en la pila
    in al, dx
    mov bx, 1 ; se coloca 1 en BX para hacer la mascara
    shl bx, cl ; se recorre a la izquierda CL veces

    or ax, bx ; se aplica la mascara en AX
    out dx, al
    pop bx ; se restaura el valor de BX
    ret
ENDP

clearBitPort PROC
    push bx ; se guarda BX en la pila
    in al, dx
    mov bx, 1 ; se coloca 1 en BX para hacer la mascara
    shl bx, cl ; se recorre a la izquierda CL veces
    not bx ; se niega BX para tener en 0 el bit que queremos apagar

    and ax, bx ; se aplica la mascara en AX
    out dx, al
    pop bx ; se restaura el valor de BX
    ret
ENDP

notBitPort PROC
    push bx ; se guarda BX en la pila
    in al, dx
    mov bx, 1 ; se coloca 1 en BX para hacer la mascara
    shl bx, cl ; se recorre a la izquierda CL veces

    xor ax, bx ; se aplica la mascara en AX
    out dx, al
    pop bx ; se restaura el valor de BX
    ret
ENDP

; - - - - -
printbin PROC
    push ax
    push cx
    mov ah, al
    mov cx, 8
```



```
@@ciclo:
    shl ah, 1
    jnc @@cero

    mov al, '1'
    jmp @@imprimir

@@cero:
    mov al, '0'
@@imprimir:
    call putchar
    loop @@ciclo

pop cx
pop ax
ret
ENDP

putchar PROC
    push ax
    mov dl, al
    mov ah, 2h
    int 21h

    pop ax
    ret
ENDP
;-----
salto PROC
    push ax
    mov ah, 2
    mov dl, 13
    int 21h

    mov dl, 10
    int 21h
    pop ax
    ret
ENDP
end principal
```



Dificultades durante el desarrollo

Durante el desarrollo de la práctica, la principal dificultad se presentó en la implementación de la rutina printBin. Al inicio tuve varios problemas debido a un manejo incorrecto de la pila. El error surgía porque no respetaba el orden de los registros al realizar las operaciones: primero hacía un push ax, luego un push cx, pero al momento de recuperar los valores ejecutaba primero un pop ax, lo que provocaba inconsistencias en la ejecución.

Referencias Bibliográficas

- [02. NASMx86: partes del código fuente](#)
- [Guide to x86 Assembly](#)
- [Using as - Assembler Directives](#)
- [NASM Assembly Language Tutorials - asmtutor.com](#)
- [Assembly - System Calls](#)